ATT-01

Autonomous Target-Tracking Platform

(ATT = Autonomous Target Tracker)

An Engineering Case Study

Real-time vision-guided target tracking: computer vision, state estimation, closed-loop control, and embedded systems

David Gabrick
Mechanical Engineering, University of Texas at Arlington
Summer 2026

Executive Summary

ATT-01 (Autonomous Target Tracker) is a self-contained pan-tilt platform that detects a designated target with a deep-learning object detector, estimates its motion with a Kalman filter, and drives two servos through a closed control loop to keep the target centered in frame. A laptop runs the computer-vision and control pipeline and streams angle commands over Wi-Fi to an ESP32 microcontroller that drives the servos.

Many hobby projects can track a target. What sets this one apart is that it was engineered and characterized like a real control system. During closed-loop bring-up, the platform exposed six distinct control failures in sequence: a derivative kick, two inverted feedback signs, a Ziegler-Nichols-characterized limit cycle, a Kalman ego-motion regression, and a camera-resolution-induced slew saturation. Each one was found in logged data, traced to a root cause, fixed, and verified. The final loop is stable, holds lock on 99 to 100 percent of frames, and is delay-limited, a conclusion backed by measured step-response data.

The finished system brings together embedded firmware, computer vision, state estimation, control theory, and mechanical design in one platform. It is accompanied by a written record of how the system was debugged and validated.

1. Motivation and Objectives

The goal was to build a portfolio project that demonstrates end-to-end systems-engineering capability for roles in defense and aerospace, where vision-guided actuation, real-time control, and embedded integration are directly relevant.

The project set out to:

- Detect and follow a specific target (a person) in real time, holding lock even when the target stops moving.
- Close a real feedback loop from camera to actuator, and tune it with engineering rigor rather than trial and error.
- Instrument the system so every design decision could be justified with measured data.
- Integrate the full stack, from optics and vision through estimation, control, firmware, and a 3D-printed mechanical assembly, into one working platform.

> *I built this because I want to work on vision-guided control systems in defense and aerospace, the kind of work done at Lockheed Martin, Northrop Grumman, L3Harris, and Bell. My interest started with photography, where I got curious about how stabilized camera gimbals hold a shot steady and keep a subject locked in frame. This platform is my hands-on way into that problem: the same detection, estimation, and closed-loop control that keep a gimbal locked on its subject.*

2. System Architecture

The platform splits work between a host laptop (perception and control) and an ESP32 microcontroller (actuation), connected over Wi-Fi:

USB webcam > YOLOv8n detection > ByteTrack persistent ID > raw centroid > Kalman filter (smoothed position and velocity) > per-axis PID > UDP over Wi-Fi > ESP32 > 50 Hz servo PWM > MG996R (pan) and SG90 (tilt).

Key design choices

- UDP over Wi-Fi for low-latency, fire-and-forget host-to-microcontroller commands.
- Detector-based tracking (YOLOv8n plus ByteTrack) instead of motion subtraction, so the platform follows a designated target through clutter and brief occlusion and does not lose a stationary subject.
- A dedicated 5 V / 3 A servo supply with common ground (never the microcontroller's 3.3 V pin), flyback diodes, and decoupling capacitors, with power integrity sized to the MG996R stall current.
- A custom 3D-printed pan-tilt bracket. Total paid bill of materials is about $45.

3. Control System Design

The host computes the pixel error between the target's estimated position and frame center, and an independent PID controller per axis converts that error into incremental servo-angle commands. Several features make the loop robust:

- Independent per-axis gains. Pan (an MG996R swinging the whole camera mass) and tilt (a light SG90) are physically different plants with different inertia, so they are tuned separately rather than sharing one gain set.
- Constant-velocity Kalman filter. The PID is driven by a filtered state estimate rather than the raw, jittery detection. Fusing a motion model with the measurement rejects bounding-box noise and lets the platform coast through brief detection dropouts.
- Slew-rate limit. The per-frame command is capped so no single command can slam a servo into its stop, and so inertial overshoot is bounded.
- Derivative-kick protection. The derivative term seeds its history on the first frame after acquisition, so the step from zero to full error cannot spike the command.
- Deadband sized to the actuator. The rest zone is set just above one command-quantization step so the platform locks confidently instead of dithering.

4. Bringing the Loop to Life: Six Failures, Diagnosed from Data

The most instructive part of the project was closed-loop bring-up. Before assembly, the software chain passed every bench test, but the servos were unmounted, so the camera never moved in response to its own commands. That is an open loop. Once the camera was bolted on and the loop truly closed, a sequence of faults appeared that an open-loop test cannot reveal. Each one is presented below as symptom, root cause, and fix.

4.1 Servo slams to its end stop on target acquisition

Symptom: The instant the detector locked on, the pan servo snapped about 90 degrees into a hard stop.

Root cause: A derivative kick. On acquisition the error steps from 0 to its full value in one frame, and the derivative term divides that step by the tiny frame time, producing a one-frame command spike of about 59 degrees that overwhelms everything else. Flipping the feedback sign, the obvious first guess, produced no change, which correctly ruled the sign out.

Fix: Seed the derivative's history on the first frame after acquisition so its computed rate is zero, clamp the loop timestep, and add a hard slew-rate limit. The slew limit also turned later faults from violent blurs into slow, readable motion that could be diagnosed.

4.2 Tracker drives the target out of frame

Symptom: With the slam gone, the camera panned so as to push the subject out of frame, then ran to its travel limit.

Root cause: An inverted feedback sign for the as-built camera orientation. The result is positive feedback, so error grew instead of shrinking. This is the sign the open-loop bench test could not have validated.

Fix: Correct the pan direction. One variable changed at a time, leaving the tilt axis untouched so every result stayed attributable.

4.3 Sustained oscillation when the subject stands still

Symptom: With direction correct, a stationary subject produced a clean, constant-amplitude pan oscillation (about plus or minus 400 px at roughly 0.77 Hz) that never decayed.

Root cause: A delay-induced limit cycle: loop transport delay plus a gain above the stability threshold. A non-decaying oscillation at a fixed gain is the Ziegler-Nichols ultimate-gain condition, so the failure itself yielded the tuning constants (Ku about 0.04, Tu about 1.3 s). Only pan oscillated, because it carries far more rotational inertia than the light tilt axis.

Fix: Back the gain below Ku and add derivative damping, a Ziegler-Nichols-informed PD design suited to a delay-dominated plant.

4.4 Distance-dependent hunting

Symptom: After further tuning, pan hunted more the farther away the subject stood, and appeared inactive up close.

Root cause: The derivative term amplifying detector noise. A smaller, farther target has a noisier bounding box, and the D term converted that jitter into full-rate servo motion. Up close,

the box is clipped by the frame edges, so its centroid pins near center and the controller correctly rests. That is real behavior, not a fault.

Fix: Reduce derivative action and low-pass the centroid error before it reaches the controller. This is standard practice for vision servoing, and it set up the Kalman filter that followed.

4.5 Kalman filter regression: the ego-motion lesson

Symptom: Adding a Kalman filter with predictive lead made tracking worse, not better. Steady-state error roughly tripled.

Root cause: Ego-motion. In a closed loop the measured centroid also moves when the camera pans, so the filter read the camera's own motion as target velocity. A stationary subject logged hundreds of px/s of phantom velocity. The predictive lead then projected that fiction forward into the command, which is positive feedback. An offline open-loop test did not reproduce it, and that is what located the cause in the closed loop.

Fix: Damp the velocity estimate hard and disable predictive lead until camera motion is compensated. The filter then serves as a position smoother and a coast-through-dropout mechanism, verified to roughly halve measurement noise.

4.6 The hidden variable: camera resolution drove slew saturation

Symptom: A stationary subject still produced a large, slow oscillation, and reducing the gain, even to a quarter of Ku, never helped.

Root cause: The camera was delivering a frame about 1280 px wide, not the roughly 640 px assumed when sizing the gains. That doubled every pixel error and pushed the command permanently past the slew-rate limit, producing a bang-bang oscillation. Bang-bang is gain-independent, which is why lowering the gain did nothing. The detected-frame error reached plus or minus 692 px, impossible inside a 640 px frame, which exposed the true resolution.

Fix: Force the capture to 640x480 and use a single-frame buffer. Maximum error halved, the command stayed under the slew limit, and the loop became linear. Slew saturation dropped from 68 percent of frames to 1 percent, and steady-state error fell from 333 px to 58 px RMS.

One practical note: the tilt servo had been physically unplugged for the entire tuning history, so every earlier observation that tilt was stable was an artifact of a disconnected actuator. Once connected, tilt showed the same inverted-sign fault as Issue 2 and was corrected the same way.

5. Results and Performance

The control story is clearest in three figures: an uncontrolled oscillation before tuning, a stable lock after, and a measured step response that characterizes the tuned loop.

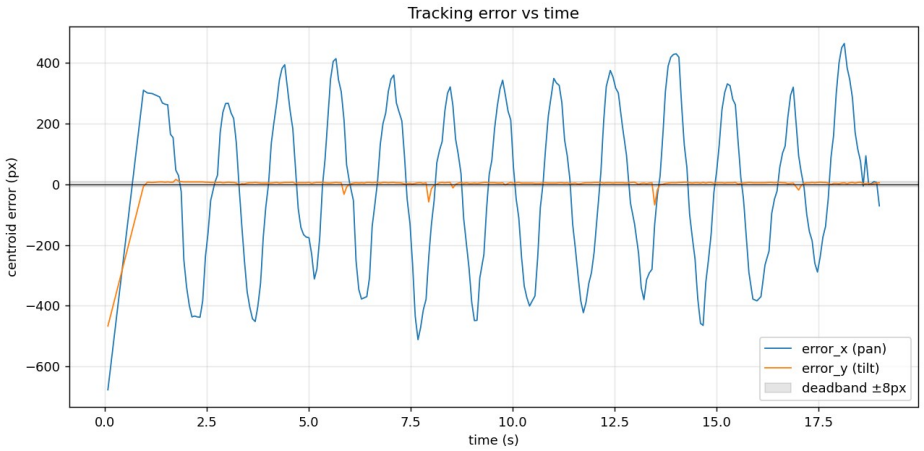

Figure 1. Before tuning: pan (blue) is a constant-amplitude limit cycle that never settles, while tilt (orange) sits flat.

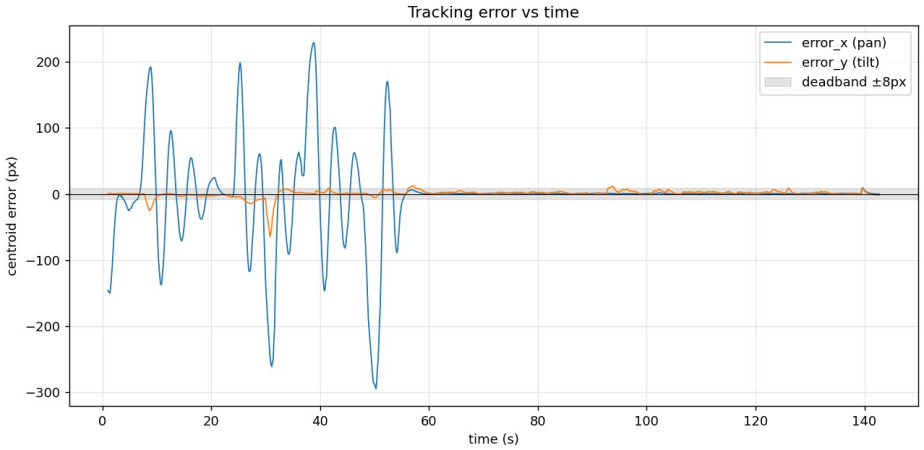

Figure 2. After the resolution and saturation fix: both axes settle into the deadband once the subject stops moving.

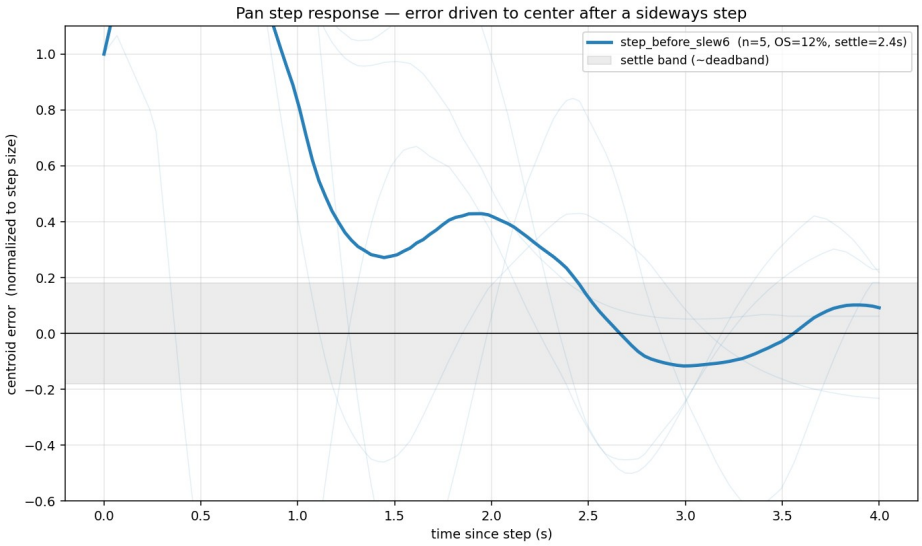

Figure 3. Ensemble pan step response from sideways-step tests. Faint traces are individual steps and the bold line is the mean: about 12 percent overshoot, 1.6 s rise, and 2.4 s settling, taken from measured data.

Measured performance

Metric	Value
Target lock rate	99 to 100 percent of frames
Control-loop rate	about 15 Hz (YOLOv8n, 480 px inference, laptop)
Steady-state tracking error (pan)	about 40 to 55 px RMS
Step response (pan)	about 12 percent overshoot, 1.6 s rise, 2.4 s settle
Ziegler-Nichols constants (pan)	Ku about 0.04, Tu about 1.3 s
Slew saturation (1280 px to 640 px fix)	68 percent to 1 percent of frames
Steady-state error (1280 px to 640 px fix)	333 px to 58 px RMS

One conclusion is worth stating plainly: the residual step overshoot does not respond to further gain or filter tuning, because the loop is now delay-limited. Two controlled experiments, one varying the derivative gain and one varying the Kalman smoothing, left the overshoot essentially unchanged, which isolates the cause to loop latency and servo inertia. This is a characterization result rather than a shortcoming, and it points to the upgrade that would help next: lower latency or a predictor.

6. Lessons Learned

1. A passing open-loop test proves the data path, not stability. Every closed-loop fault was invisible on the bench.
2. A negative result is information. Flipping the sign and seeing no change is what ruled it out and pointed at the real cause.
3. Make failures observable before interpreting them. The slew-rate limit did not fix the sign error, but it slowed the motion enough to diagnose it.
4. Let the system reveal its own constants. The standstill oscillation doubled as a Ziegler-Nichols experiment that handed over Ku and Tu directly.

5. Sensor configuration is part of the control loop. Resolution, frame rate, and buffering set loop gain and delay as surely as the controller gains do.
6. A model-based estimator is only as good as its model. The Kalman filter's constant-velocity model was right for the target but blind to the camera's own motion.

> *The most surprising moment was adding a Kalman filter and watching tracking get worse instead of better, until I saw it was reading the camera's own panning as target motion and chasing a ghost. The hardest was the oscillation that would not go away no matter how far I dropped the gain, which turned out to be the camera quietly feeding me a 1280 px frame instead of the 640 px I had sized everything around. And the most humbling was realizing the tilt servo had been unplugged the whole time, so every result I had trusted for that axis meant nothing. Across all three, the lesson was the same: trust the logged data over my own assumptions, and look upstream of the obvious suspect before reaching for a more complex fix.*

7. Future Work

- Lower latency. Move inference to a GPU or Jetson, or use a smaller model, to push the loop past 15 Hz. Because the overshoot is delay-limited, halving latency roughly halves overshoot.
- Sub-degree actuation. Command the servos in fractional degrees (microsecond PWM) to roughly triple angular resolution and shrink the deadband without re-introducing dither.
- Ego-motion-compensated prediction. Subtract the camera's commanded angular rate from the estimated target velocity, then re-enable predictive lead to anticipate motion instead of reacting to it.
- Mechanical analysis. Servo-torque and inertia calculations, plus finite-element analysis on the bracket, to formalize the mechanical design.

Appendix A. Final Configuration

Parameter	Pan	Tilt
Proportional gain (Kp)	0.018	0.020
Derivative gain (Kd)	0.008	0.004
Integral gain (Ki)	0	0
Deadband	22 px (about 2 deg)	22 px
Slew limit	4 deg per frame	4 deg per frame

Subsystem	Detail
Detector	YOLOv8n (person class), 480 px inference
Tracker	ByteTrack persistent ID
Estimator	Constant-velocity Kalman filter (position smoother and coast)
Capture	640x480, single-frame buffer
Link	UDP over 2.4 GHz Wi-Fi to ESP32
Actuators	MG996R (pan), SG90 (tilt), 5 V / 3 A supply

A full record of every fault and fix is kept in the project's control-systems tuning log. All figures are reproduced from logged run data using the project's own plotting tools.